Recursion in C++

Infinite Logic through Self-Reference

Dividing Complexity into Simplicity

BASE CASE • RECURSIVE CASE • STACK LOGIC

1. The Self-Calling Loop

Recursion is a programming technique where a function calls itself. It is a mathematical way of solving a problem by breaking it down into smaller, similar sub-problems.

> "To understand recursion, you must first understand recursion."

When to use Recursion:

- When the problem has a "fractal" nature (the small part looks like the whole).
- For complex data structures like **Trees** and **Graphs**.
- For algorithmic challenges like **Searching**, **Sorting**, and **Backtracking**.

> **Analogy:** Think of a set of **Russian Matryoshka dolls**. To get to the smallest doll, you must open a big one, then a slightly smaller one, then another, until you reach the core.

2. Anatomy of a Recursive Function

A recursive function without a plan is a disaster. Every valid recursive function consists of two parts:

1. The Base Case (The Stop Sign)

This is the condition that stops the recursion. Without this, the function will call itself forever until the computer crashes.

2. The Recursive Case (The Step)

This is where the function calls itself, but with a **smaller input**. This ensures that the program eventually reaches the base case.

```
void count(int n) {
    if (n == 0) return; // 🚩 BASE CASE

    cout << n;
    count(n - 1);       // 🔁 RECURSIVE CASE
}
```

3. Behind the Scenes: The Call Stack

Recursion doesn't happen all at once. C++ uses a **Call Stack** to manage these self-calls.

The Process:

1. **Pushing:** Each time the function calls itself, a new "layer" is added to the stack in memory. The current state is "paused."
2. **Reaching Bottom:** The computer keeps adding layers until the **Base Case** is triggered.
3. **Unwinding:** Once the base case is hit, the computer starts finishing the paused functions from top to bottom.

> **Stack Overflow:** If the recursion is too deep (too many layers), the "Stack" runs out of memory and the program crashes.

4. Practical Example: Sum of K

Let's calculate the sum of all numbers from k down to `0`.

```
int sum(int k) {
    if (k > 0) {
        return k + sum(k - 1);
    } else {
        return 0; // Base Case
    }
}
```

Execution Trace for sum(3):

- `sum(3)` returns `3 + sum(2)`
- `sum(2)` returns `2 + sum(1)`
- `sum(1)` returns `1 + sum(0)`
- `sum(0)` returns `0` (Base Case hit!)
- Unwinding: `1 + 0 = 1` → `2 + 1 = 3` → `3 + 3 = 6`

5. The Classic Factorial

In mathematics, the factorial of n (written as n!) is the product of all positive integers less than or equal to n.

```
int factorial(int n) {
    if (n > 1) {
        return n * factorial(n - 1);
    }
    return 1; // Base case for 1! and 0!
}
```

Logic: To find the factorial of 5, you find `5 * (factorial of 4)`. To find that, you find `4 * (factorial of 3)`, and so on.

6. Recursion vs. Loops

Almost any recursive problem can be solved with a `while` or `for` loop. Why choose one over the other?

Feature	Recursion	Iteration (Loops)
Code Style	Elegant, cleaner logic	Longer, more variables
Performance	Slower (Func call overhead)	Faster
Memory	High (Stack usage)	Low (O(1) memory)
Best for	Trees, Graphs, Sorting	Simple repetition

7. Rules for Safe Recursion

1. Validate your Base Case

Ensure the base case is reachable. If you count `n-1` but your base case checks if n is exactly `-100`, you might skip it and loop forever.

2. Move Toward the Base Case

Your argument must change in every call to get closer to the exit condition. (e.g., `n - 1` or `n / 2`).

3. Think of Memory

For large inputs (like 100,000 calls), recursion is usually the wrong tool unless you use "Tail Call Optimization."

8. Summary & Final Challenge

```
// SUMMARY CHECKLIST:
// 1. Function calls itself? ✅
// 2. Smaller input provided? ✅
// 3. Exit condition defined? ✅
```

Final Skill Challenge:

The Fibonacci Sequence:

The sequence starts: 0, 1, 1, 2, 3, 5, 8, 13...

Each number is the sum of the two before it. Write a recursive function `int fib(int n)` that returns the Nth Fibonacci number.

Hint: The base case should handle when n is 0 or 1.

MODULE COMPLETE: RECURSIVE ARCHITECTURE